COROS Partner API — Signature Verification & Daily Data Push Cheat Sheet

For credentialed Partner API partners Last updated: October 21, 2026 | API Reference V2.1.1

How Push Verification Works

When COROS pushes data to your endpoint (Workout Summary Push 5.4, or Daily Data Push 5.5), the request includes **two layers of verification**:

1. **Header credentials** — `client` and `secret` in the HTTP request headers
2. **URL signature** — `signature`, `nonce`, and `timestamp` as URL query parameters

Your endpoint must verify both.

Layer 1: Header Credentials

COROS sends your `client` and `secret` (assigned during onboarding) in the request headers.

Header	Value
`client`	Your client_id (assigned by COROS)
`secret`	Your client_secret (assigned by COROS)

Verify: Compare the header values against your stored credentials. Reject if they don't match.

Layer 2: URL Signature (SHA-1)

COROS appends three query parameters to your callback URL:

```
POST https://your-endpoint.com/webhook?signature=xxx&timestamp=xxx&nonce=xxx
```

Parameter	Type	Description
`signature`	String	SHA-1 hex hash for verification
`timestamp`	String	Unix timestamp (e.g. `1715428513`)
`nonce`	String	Random value (e.g. `321291201`)

Verification Algorithm

Step 1: Take three values: your **token**, the **timestamp**, and the **nonce**.

> Your **token** is the 16-32 character alphanumeric string you submitted to COROS when registering your endpoint.

Step 2: Sort these three strings in **lexicographic (natural string) order**.

Step 3: Concatenate the sorted values into a single string (no separator).

Step 4: Compute the **SHA-1 hex digest** of that string.

Step 5: Compare the result with the `signature` query parameter. A match confirms the request is from COROS.

Code Examples

Java

```
import org.apache.commons.codec.digest.DigestUtils;

// SIGN_TOKEN is the token you submitted to COROS (16-32 alphanumeric chars)
private static final String SIGN_TOKEN = "s1ntjN9GkiMKeygoH3r84pC2dc05TvWI";

public boolean checkSignature(String signature, String timestamp, String nonce) {
    // Sort timestamp, nonce, and token in natural string order
    List<String> values = Arrays.asList(nonce, timestamp, SIGN_TOKEN);
    Collections.sort(values);

    // SHA-1 hex of concatenated sorted values
    String calculated = DigestUtils.sha1Hex(
        values.get(0) + values.get(1) + values.get(2)
    );

    return Objects.equals(calculated, signature);
}
```

PHP

```
$signature = $_GET["signature"];  // e.g. f542618b60ce0211349a7a0d5edfd347ef903b5a
$timestamp = $_GET["timestamp"];  // e.g. 1715428513
$nonce     = $_GET["nonce"];      // e.g. 321291201
$token     = TOKEN;               // Your registered token

$parameters = array($token, $timestamp, $nonce);
sort($parameters, SORT_STRING);
$value = implode($parameters);
```

```
$calculated = sha1($value);

return $calculated == $signature;
```

Python

```
import hashlib

SIGN_TOKEN = "s1ntjN9GkiMKeygoH3r84pC2dc05TvWI"  # Your registered token

def check_signature(signature, timestamp, nonce):
    values = sorted([SIGN_TOKEN, timestamp, nonce])
    calculated = hashlib.sha1("".join(values).encode("utf-8")).hexdigest()
    return calculated == signature
```

Node.js

```
const crypto = require("crypto");

const SIGN_TOKEN = "s1ntjN9GkiMKeygoH3r84pC2dc05TvWI"; // Your registered token

function checkSignature(signature, timestamp, nonce) {
    const values = [SIGN_TOKEN, timestamp, nonce].sort();
    const calculated = crypto
        .createHash("sha1")
        .update(values.join(""))
        .digest("hex");
    return calculated === signature;
}
```

Worked Example

Inputs: - Token: `s1ntjN9GkiMKeygoH3r84pC2dc05TvWI` - Timestamp: `1715428513` - Nonce: `321291201`

Step 1 — Sort lexicographically:

```
["1715428513", "321291201", "s1ntjN9GkiMKeygoH3r84pC2dc05TvWI"]
```

Step 2 — Concatenate:

```
1715428513321291201s1ntjN9GkiMKeygoH3r84pC2dc05TvWI
```

Step 3 — SHA-1 hex:

```
f542618b60ce0211349a7a0d5edfd347ef903b5a
```

Step 4 — Compare with the `signature` query parameter. Match = valid.

Daily Data Push (Section 5.5) — Payload Reference

COROS pushes each connected user's most recent **three days** of daily data.

Request Format

```
POST https://your-endpoint.com/daily?signature=xxx&timestamp=xxx&nonce=xxx

Headers:
  client: {your_client_id}
  secret: {your_client_secret}

Body (JSON):
```

Payload Fields

Field	Type	Description	Format
`batchDailyList`	List	Top-level array (multiple users per batch)	
`openId`	String	User identifier	
`dailyList`	List	Daily records for this user	
`happenDay`	32-bit int	Date of the data	`yyyyMMdd` (e.g. `20200615`)
`sleepStartTime`	String	Sleep start (time in bed)	`yyyy-MM-dd HH:mm:ss`
`sleepEndTime`	String	Sleep end (time in bed)	`yyyy-MM-dd HH:mm:ss`
`calorie`	Float	Calories (kcal)	
`step`	32-bit int	Total steps	
`rhr`	32-bit int	Resting heart rate (bpm)	Omitted when unavailable
`hrvList`	List	Raw HRV time series for the day	
`hrv`	32-bit int	Individual HRV value	
`timestamp`	32-bit int	Unix timestamp of HRV reading	

Field	Type	Description	Format
`hr`	32-bit int	Heart rate at that reading	
`ppgHrv`	32-bit int	Average HRV assessment (RMSSD, ms)	Overnight average
`sleepAvgHr`	32-bit int	Average heart rate during sleep	

Example Payload

```
{
  "batchDailyList": [
    {
      "openId": "a1dda895324c4c0497d8bed8b9dbbd02",
      "dailyList": [
        {
          "happenDay": 20200615,
          "sleepStartTime": "2020-06-14 22:00:01",
          "sleepEndTime": "2020-06-15 08:00:01",
          "calorie": 9553,
          "step": 52,
          "rhr": 56,
          "hrvList": [
            { "hrv": 25, "hr": 60, "timestamp": 1592098222 },
            { "hrv": 30, "timestamp": 1592101822 }
          ],
          "ppgHrv": 50,
          "sleepAvgHr": 70
        }
      ]
    }
  ]
}
```

Your Response

Return this to acknowledge receipt:

```
{
  "result": "0000",
  "message": "ok"
}
```

`result: "0000"` = success. Any other value = COROS treats it as failed and retries. If push fails for more than 24 hours, the affected data will no longer be pushed.

Key Notes

- **sleepStartTime / sleepEndTime** represent **time in bed**, not time asleep. Total sleep time and sleep stages are not available in the API at this time.
- **ppgHrv** is RMSSD measured in milliseconds (overnight average).
- **hrvList** contains raw time-series HRV values throughout the day, not just the overnight average.
- **Same URL for both pushes:** You can use the same endpoint URL for Workout Summary Push and Daily Data Push. Differentiate by the payload structure (`sportDataList` vs `batchDailyList`).
- **Same credentials:** Daily Data Push uses your existing client_id and client_secret. No separate DDP credentials are issued.
- **Token:** Submitted during endpoint registration. This is different from your client_id/secret. It is used only for signature verification.

Questions?

developer@coros.com